

DI-725 : Transformers and Attention-Based Deep Networks

Assignment - 1

Turgay Yıldız
Graduate School of Informatics
Middle East Technical University
Ankara, Turkey
yildiz.turgay@metu.edu.tr

Abstract—This report presents training of a nanoGPT model and the use of a pre-trained GPT-2 model for sentiment classification. The dataset was preprocessed using tokenization, stemming, lemmatization, and stopword removal, while exploratory data analysis was performed through various visualizations. Key modifications to the nanoGPT architecture—such as adding a classification head and tuning hyperparameters—enabled effective classification. Evaluation using metrics like accuracy, loss, and confusion matrices demonstrated the benefits of fine-tuning a pre-trained model over training from scratch. Experiment tracking with WANDB provided insights into hyperparameters, classification accuracies, and overall model behavior.

Index Terms—Transformers, Nano-GPT, Pre-trained, GPT-2, Transfer Learning

I. INTRODUCTION

This project focuses on training a nanoGPT model and leveraging a pre-trained GPT-2 model for sentiment classification in an NLP task. The training dataset comprises two columns—conversation and sentiment—with imbalanced class distributions, making generalization to unseen data challenging. Preprocessing steps, including tokenization, lemmatization, and stopword removal, were applied. Labels were then encoded numerically, and a pre-trained tokenizer was used to convert conversations into tokens. Finally, tokens and labels were transformed into tensors for training. The nanoGPT model originally ends with a linear layer; hence, a new classification head was implemented for sentiment prediction.

For the second part of the project, I used the pre-trained GPT-2 weights. I initially froze most layers—training only the last layer and the classification head—and later unfroze all layers to fine-tune with a very small learning rate. Due to the small and imbalanced dataset, performance issues arose, leading to modifications that penalized errors on the less frequent positive class more heavily. These details and results are elaborated further in the report.

II. DATASET

The training dataset consists of 11 columns and 970 rows. I dropped nine columns, retaining only the sentiment labels

and conversations. There are three sentiment classes: neutral (542 instances), negative (411 instances), and positive (17 instances). The extreme imbalance (Fig. 1) —especially for the positive class—led to many misclassifications, with the model often confusing *positive* with *neutral* sentiments. Notably, the most frequent class (neutral) represents about 56% of the data, which serves as a baseline.

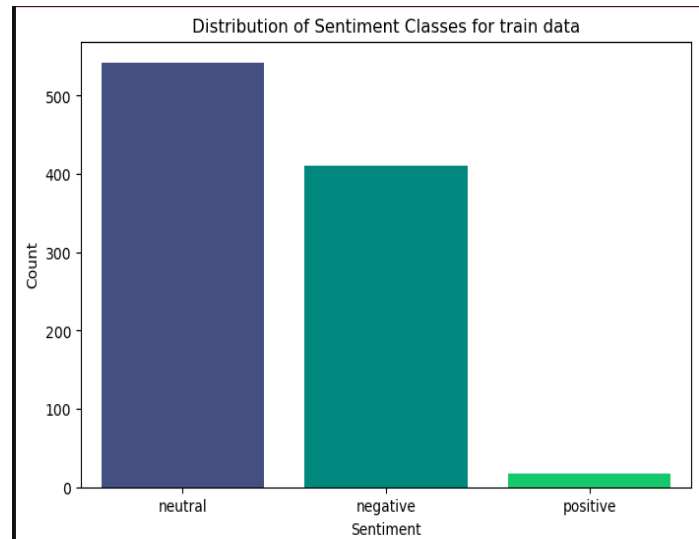


Fig. 1. Results

Exploratory Data Analysis (EDA) confirmed the absence of *missing values*. Two columns showed a very high *correlation* (Fig. 2) (0.98) and two others a moderate correlation (0.62); however, these did not affect the analysis as they were excluded from the final dataset. For preprocessing, I converted POS tags to WordNet format to improve lemmatization. Conversations were *tokenized*, *lemmatized*, and cleansed of *stopwords*, and labels were encoded numerically. A pre-trained GPT2Tokenizer was used with a default *padding* length of 512, which was later adjusted to

1024 without significant changes to performance.

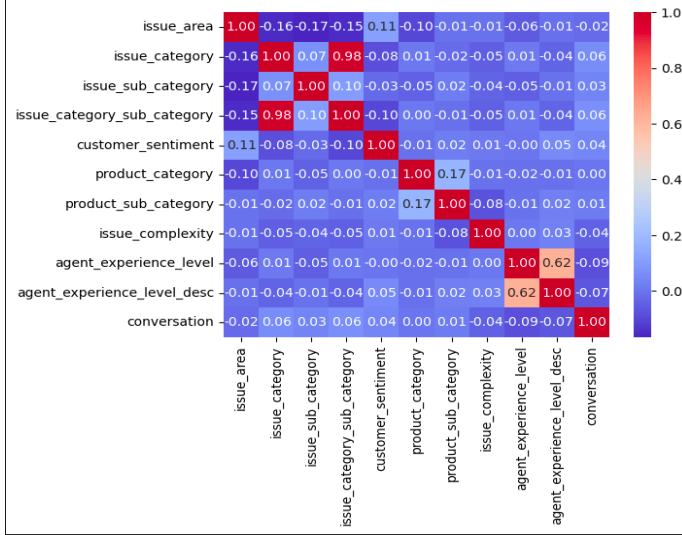


Fig. 2. Results

III. MODELING

First, I modified Nano-GPT by adding a classification head. I experimented with various hyperparameters, such as different learning rates and layer configurations. After initial training, it became clear that most errors arose from the underrepresented positive class. To address this, I revised the classification head to penalize errors on this class more severely, taking care to handle any dimensionality issues.

Next, I fine-tuned the pre-trained GPT-2 model. Since the hyperparameters of the pre-trained model could not be changed, I only modified the input dimension of the linear layer in the classification head to match the vocabulary size. The training strategy involved freezing all layers except the last layer and the classification head, followed by unfreezing all layers and training with a very small learning rate (0.0000001 instead of 0.0001).

Both models were trained exclusively on the train data, and given the small and imbalanced nature of the dataset, the reported accuracy rates should be interpreted with caution. The code is available on my GitHub repository [2].

IV. EVALUATION

As reflected in WANDB [1] tracking, the initial training of both models was performed solely on the train data. However, to monitor overfitting, I allocated a small subset of the train data for validation. This approach enabled a better assessment of overfitting. one of the model runs, which overfitted, ended up with an accuracy of 57%. In contrast, another run—despite having higher training and validation losses—managed to converge at 77% accuracy. After modifying the error penalization, the pre-trained model reached an accuracy of 83%. Although some training runs showed very low loss values despite a

dropout rate of 0.2—indicating overfitting—others converged with moderate loss values. The *confusion matrices* revealed that most errors occurred in the *positive* sentiment class, with frequent misclassification as *neutral*.

V. RESULTS

Experiment results (Fig. 3) were meticulously tracked using WANDB [1], which provided detailed documentation of training progress and performance metrics. The fine-tuned pre-trained model demonstrated superior convergence and final accuracy compared to the model trained from scratch. Notably, increasing the max padding length from 512 to 1024 (last two runs) did not yield significant differences in performance.

loss	Train L	Val Los	GPT-2:	GPT-2:	LR	acc
-	0.10997	0.2376	-	-	0.0001	-
-	0.070165	0.054212	-	-	0.0001	-
-	-	-	0.01144	0.036729	0.00001	0.83333
-	-	-	0.030414	0.0002456	0.00001	0.7
-	-	-	0.0031045	0.0000128	0.000001	0.7
-	0.0000046	0.0000026	-	-	-	0.56667
-	0.0006304	0.82516	-	-	-	0.76667
0.084388	-	-	-	-	-	0.63333
0.10184	-	-	-	-	-	0.63333

Fig. 3. Results

VI. DISCUSSION

Comparing fine-tuning with training from scratch revealed significant advantages for the former. Fine-tuning a pre-trained model leverages prior knowledge, reducing training time and computational requirements while delivering better performance. In contrast, training from scratch offers more control over the learning process but requires extensive resources and more data. These findings underscore the efficiency and practicality of transfer learning, particularly for tasks with limited data.

VII. CONCLUSION

The study confirms that fine-tuning a pre-trained GPT model is an effective strategy for improving performance, reducing training time, and achieving robust results. Thorough dataset preprocessing, strategic model modifications, and comprehensive evaluation contributed to the success of this approach, highlighting the practical benefits of transfer learning in modern NLP applications.

REFERENCES

- [1] WANDB: https://wandb.ai/DI_725_Assignment_1/DI_725_Assignment_1___2697258.
- [2] GitHub: <https://github.com/turgay-yildiz/DI-725-Transformers-and-Attention-Based-Deep-Networks-/tree/main>.